

FRET 组合式可实现性分析算法复现与连通分量优化

1. 引言

本次期末大作业基于前期课程汇报论文 Mavridou 等人在 FM 2021 发表的 From Partial to Global Assume-Guarantee Contracts: Compositional Realizability Analysis in FRET。该论文研究形式化需求中的可实现性检查问题，并将全局 assume-guarantee contract 拆分为多个更小的 partial contracts，从而缓解单体验证带来的规模和调试困难。

选择该论文的原因有三点。第一，论文中的核心算法是明确的图算法：根据需求涉及的状态变量或输出变量建立需求之间的连通关系，再求 connected components。第二，仓库中已有 FRET 导出的需求数据和可运行的最小化复现框架，便于复现实验。第三，该算法虽然简单清晰，但其连通分量构造过程仍有可优化空间，适合围绕算法复杂度做小规模改进。

本文主要完成三项工作：

1. 复现论文中基于 connected components 的组合式分解算法。
2. 在 LMCPS/FRET 数据集上展示复现结果。
3. 提出并实现一种基于输出变量倒排索引和并查集的连通分量优化算法，并通过实验分析其有效性和局限。

本文参考了 CompositionVerification/docs/FM_2026_paper_205.pdf 中的 dependency-enhanced compositional verification 思路，但没有采用该论文的数据依赖、设备状态依赖和 SCC 分组路线；改进点聚焦于 Mavridou 2021 原算法本身的 connected-component 阶段。

2. 问题定义

论文研究的是反应式系统需求的可实现性检查。给定一个 assume-guarantee contract (A, G) ，其中 A 是环境假设， G 是系统保证。可实现性要求存在一个系统实现，使得对所有满足 A 的环境输入，系统都能选择后继状态并持续满足 G 。

在 FRET

生成的需求模型中，每条需求会被翻译为形式化属性。对一个组件而言，输入可以抽象为：

- 需求集合 $R = \{r_1, r_2, \dots, r_n\}$ 。
- 变量集合 V ，包括输入变量、输出变量和内部变量。
- 每条需求 r_i 依赖的输出变量集合 $Out(r_i)$ 。

输出目标是将需求集合划分为若干子集：

$$P = \{C_1, C_2, \dots, C_k\}$$

其中每个 C_i 是一个 connected component，对应一个 partial contract。若两个需求直接或间接共享输出/状态变量，则它们必须位于同一分量中，以避免局部实现之间出现互相干扰。

评价指标包括：

- 分量数量 k 。
- 最大分量大小 $\max |C_i|$ 。
- 分解结果是否与 baseline 等价。

- 分解阶段的运行时间。

3. 算法原理

3.1 原论文算法

Mavridou 2021

的核心思想是：如果多个需求之间没有共享状态变量，则它们可以被拆成互不干扰的 `partial contracts` 独立验证。算法流程可以概括为：

1. 从 FRET 导出的需求中提取每条需求依赖的输出变量。
2. 若两条需求依赖的输出变量集合存在交集，则认为它们属于同一连通区域。
3. 对需求图求 `connected components`。
4. 为每个 `connected component` 生成一个局部 `Lustre contract`。
5. 分别对局部 `contract` 进行 `realizability checking`。

伪代码如下：

```
Input: requirements R, output dependency map Out
Output: connected components C

C = []
for each requirement r in R:
    S = Out(r)
    find a component c in C such that c.outputs intersects S
    if such c exists:
        add r to c
        c.outputs = c.outputs union S
    else:
        create new component {r} with outputs S
merge components that still share outputs
add assumption requirements to every component
return C
```

该方法的优点是实现简单，且分解语义直观：共享输出变量的需求不会被拆开。

3.2 本文优化算法

原实现每加入一条需求，都可能和已有分量逐个比较输出集合交集。当需求数量较多且大部分需求互不相关时，这种比较会产生大量无效开销。

本文提出的优化方法是使用输出变量倒排索引和并查集：

1. 创建并查集，每条需求初始为独立集合。
2. 建立 `firstRequirementByOutput` 映射，记录每个输出变量第一次出现在哪条需求中。
3. 扫描需求 `r` 的每个输出变量 `v`：
 - 如果 `v` 第一次出现，记录 `firstRequirementByOutput[v] = r`。
 - 如果 `v` 已出现，则将 `r` 与之前出现过 `v` 的需求做 `union`。
4. 扫描结束后，按并查集根节点聚合需求，得到 `connected components`。
5. `assumption` 需求仍加入所有分量，与原方法保持一致。

优化伪代码如下：

```
Input: requirements R, output dependency map Out
Output: connected components C

make-set(r) for each r in R
```

```

first = empty map

for each requirement r in R:
    for each output v in Out(r):
        if v not in first:
            first[v] = r
        else:
            union(r, first[v])

C = group requirements by find(r)
add assumption requirements to every component
return C

```

该优化不改变分解规则，只改变 `connected components` 的计算方式。

4. 复杂度分析

设：

- n 为需求数量。
- m 为需求到输出变量的引用总数，即 $m = \sum |Out(r_i)|$ 。
- k 为平均输出集合大小。

原始实现中，每条需求可能与已有多个分量进行集合求交。最坏情况下，如果许多需求彼此独立，会形成大量已有分量，此时复杂度接近：

$O(n^2 * k)$

优化实现中，每个输出引用只被扫描一次，并查集操作的均摊复杂度接近常数：

$O((n + m) * \alpha(n))$

其中 $\alpha(n)$ 是反 Ackermann 函数，在实际规模下可视为小常数。

空间复杂度方面，优化算法需要额外维护输出变量倒排索引和并查集数组：

$O(n + |V_{out}|)$

该优化适用于分量较多、共享输出较少的稀疏需求集合。若需求高度耦合，很快合并为少数大分量，则原实现常数开销更低，优化版不一定更快。

5. 复现方法

5.1 实验环境

实验在 Windows 本地环境中运行，使用 Node.js 执行 JavaScript 复现代码。

```

node --version
# v24.14.0

```

核心目录：

- `tools/nasa-method`: Mavridou/NASA FM 2021 baseline 复现。
- `tools/optimized-nasa`: 本文新增的优化实现。
- `tools/benchmark-optimization.js`: baseline 与优化算法的 benchmark 脚本。
- `datasets/LMCPs/LM_requirements.json`: 主要实验数据。

5.2 复现命令

列出数据集组件：

```
node run.js list --dataset datasets/LMCPS/LM_requirements.json
```

运行原论文 **baseline**:

```
node run.js nasa --dataset datasets/LMCPS/LM_requirements.json
```

生成复现对比结果:

```
node run.js compare --dataset datasets/LMCPS/LM_requirements.json --out-dir
outputs/reproduction
```

运行优化算法:

```
node run.js optimized --dataset datasets/LMCPS/LM_requirements.json --component Euler
```

运行优化 **benchmark**:

```
node run.js benchmark --dataset datasets/LMCPS/LM_requirements.json --scales
1,10,50,100,500 --repeat 30 --out-dir outputs/optimization-benchmark
```

5.3 复现范围说明

由于本机未安装 `jkind/jrealizability` 和 `z3`, 本文没有完成 `solver` 层面的真实可实现性判定。当前复现范围包括:

- 读取 FRET JSON 数据。
- 构建需求到输出变量的依赖映射。
- 计算 `connected components`。
- 生成 `monolithic` 和局部 `Lustre contract` 文件。
- 对分解结果和分解阶段运行时间进行实验分析。

6. 实验结果与分析

6.1 原论文 **baseline** 分解结果

在 `datasets/LMCPS/LM_requirements.json` 上, 复现得到如下 `connected-component` 分解结果。完整结果见 `outputs/reproduction/LM_requirements/comparison.md`。

Component	Requirements	NASA CCs	Max CC Size
Autopilot	9	4	4
EB	3	2	2
Euler	20	1	20
FSM	3	1	3
FSM_Autopilot	12	1	12
FSM_Sensor	6	1	6
NLGuidance	7	7	1
NN	5	1	5
Regulator	10	5	2
RollAutopilot	8	2	6
SWIM	3	2	2
TriplexSignalMonitor	6	1	6

Component	Requirements	NASA CCs	Max CC Size
Tustin_Integrator	5	1	5

可以看到，不同组件的可分解性差异明显。NLGuidance 被拆为 7 个大小为 1 的独立分量，说明需求之间几乎没有共享输出。Euler、FSM_Autopilot、TriplexSignalMonitor 等组件只有 1 个分量，说明需求之间共享输出关系较强，无法通过该方法进一步拆分。

6.2 优化算法等价性

benchmark 共包含 13 个组件和 5 个放大倍数，共 65 组实验。所有实验均满足：

```
equivalent = true
```

这说明优化算法与 baseline 产生完全相同的需求分区，优化没有改变原论文算法语义。

6.3 优化算法性能

代表性结果如下，完整数据见

outputs/optimization-benchmark/LM_requirements/benchmark.md。

Component	Scale	Requirements	Baseline median ms	Optimized median ms	Speedup
NLGuidance	1	7	0.0041	0.0023	1.78
NLGuidance	10	70	0.1011	0.0127	7.96
NLGuidance	50	350	1.9417	0.0673	28.85
NLGuidance	100	700	7.7197	0.1368	56.43
NLGuidance	500	3500	175.3034	0.9953	176.13
Euler	100	2000	0.9933	0.9064	1.10
Euler	500	10000	7.1751	7.1872	1.00
Autopilot	500	4500	1.1252	2.7630	0.41
Regulator	500	5000	1.4581	2.0840	0.70

实验结论：

1. 对于 NLGuidance 这类高度可分解、分量很多的输入，优化效果非常明显。500 倍放大时，baseline 中位数为 175.3034 ms，优化版为 0.9953 ms，约 176 倍加速。
2. 对于 Euler 这类高度耦合组件，优化版与 baseline 基本相当。
3. 对于部分小规模或高度耦合组件，并查集和倒排索引带来的额外常数开销可能使优化版慢于 baseline。

因此，本文优化的价值主要体现在改善原算法在稀疏需求集合上的最坏趋势，而不是保证所有输入上都加速。

7. 改进思路与扩展实验

7.1 改进动机

原论文算法中的 connected-component 计算采用逐需求、逐分量比较输出集合交集的方式。该方式在小规模输入上直观可靠，但当需求数量增加且分量数量较多时，会进行大量无效的集合交叉判断。

7.2 改进方法

本文将“判断哪些需求共享输出变量”转化为倒排索引问题：

```
output variable -> requirements touching this output
```

随后用并查集合并共享输出变量的需求。这样每个输出引用只参与一次扫描，避免反复与无关分量做集合交集。

7.3 实验设计

为验证优化效果，本文使用真实 LMCPs 数据集中的 13 个组件，并通过复制需求构造

1、10、50、100、500 倍规模。评价指标包括：

- 分解结果是否与 baseline 等价。
- baseline 分解时间中位数。
- optimized 分解时间中位数。
- speedup。

7.4 结果分析

结果说明优化具有结构依赖性。对于输出共享稀疏、分量数量多的系统，倒排索引和并查集显著减少无效比较；对于强耦合系统，原算法很快合并到少数分量，优化版的额外数据结构反而成为负担。

后续可以进一步设计 hybrid 策略：

1. 先统计输出引用稀疏度和输出变量重复率。
2. 若分解预计稀疏，则选择 union-find 优化算法。
3. 若输出共享高度集中，则使用原 baseline 实现。

8. 总结

本次作业完成了对 Mavridou 2021 FRET 组合式可实现性分析算法的复现，并围绕 connected-component 计算提出了一个小规模算法优化。通过实验可以看到，原论文方法在需求可分解时能够有效产生多个 partial contracts；优化算法在不改变分解结果的前提下，显著改善了稀疏需求集合上的运行时间。

复现过程中也发现该方法存在局限。首先，组合式分解效果高度依赖需求之间是否共享输出或状态变量。若所有需求强耦合，则无法获得明显分解收益。其次，本文当前环境缺少 jkind/jrealizability 和 z3，因此没有完成 solver 层面的可实现性结果验证。最后，优化算法并非在所有组件上都更快，说明实际工具中应根据输入结构选择算法。

本次实验加深了我对图算法在形式化方法工具中的作用的理：论文中的理论分解最终落到可计算的连通分量问题，而一个看似简单的 connected-component 阶段，也可以通过数据结构选择影响实际运行效率。

9. 参考文献

[1] Anastasia Mavridou, Andreas Katis, Dimitra Giannakopoulou, David Kooi, Thomas Pressburger, Michael W. Whalen. From Partial to Global Assume-Guarantee Contracts: Compositional Realizability Analysis in FRET. FM 2021.

[2] NASA-SW-VnV. FRET: Formal Requirements Elicitation Tool.

<https://github.com/NASA-SW-VnV/fret>

[3] CompositionVerification/docs/FM_2026_paper_205.pdf, **dependency-enhanced compositional verification reference material used only for comparison of possible directions.**